

Arquitectura de computadores y sistemas operativos



Arquitectura de computadores y sistemas operativos

PowerShell

Referencias:

https://www.youtube.com/watch?v=YwGIXXqLDkM&list=PLn98b7UTDjb1h5_LCHXyeJR8nrPeTaSBM

<https://www.youtube.com/watch?v=Ej82nzcpFGk>

<https://www.youtube.com/watch?v=cDcS6iL1G4I>

https://www.youtube.com/watch?v=J9NEgkucB_s

<https://www.youtube.com/watch?v=A4Q0XTctV5M>

https://www.youtube.com/watch?v=sQm4zRvvX58&list=PLIVtbbG169nFq_hR7FcMYg32xsSAObuq8

REINVENTEMOS
EL FUTURO



PowerShell

Es una solución de automatización de tareas multiplataforma formada por un shell de línea de comandos, un lenguaje de scripting y un marco de administración de configuración. PowerShell funciona en Windows, Linux y macOS.

PowerShell es un shell de comandos moderno que incluye las mejores características de otros shells populares. A diferencia de la mayoría de los shells que solo aceptan y devuelven texto, PowerShell acepta y devuelve objetos .NET.

PowerShell es un lenguaje de scripting que se usa para automatizar la administración de sistemas. También se usa para compilar, probar e implementar soluciones, a menudo en entornos de CI/CD (Entrega y despliegue continuo). PowerShell se basa en .NET Common Language Runtime (CLR). Todas las entradas y salidas son objetos de .NET.

PowerShell

Los comandos de PowerShell se conocen como **cmdlets**. Además de los **cmdlets**, PowerShell permite ejecutar cualquier comando disponible en el sistema.

Los cmdlets son comandos de PowerShell nativos, no ejecutables independientes. **Los cmdlets** se recopilan en módulos de PowerShell que se pueden cargar a petición. **Los cmdlets** se pueden escribir en cualquier lenguaje .NET compilado o en el mismo lenguaje de scripting de PowerShell.

PowerShell usa un par de términos **verbo-nombre** para asignar nombres a los **cmdlets**. Por ejemplo, el **cmdlet Get-Command** incluido en PowerShell se usa para obtener todos los cmdlets que están registrados en el shell de comandos. El verbo identifica la acción que realiza el cmdlet y el nombre identifica el recurso en el que el cmdlet realiza la acción.

```
Windows PowerShell
PS C:\Users\MARCELOFGB> Get-help Get-Process

NOMBRE
Get-Process

SINTAXIS
Get-Process [[-Name] <string[]>] [<CommonParameters>]

Get-Process [[-Name] <string[]>] [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

ALIAS
gps
ps

PS C:\Users\MARCELOFGB> Get-help Get-Process -full

NOMBRE
Get-Process

SINTAXIS
Get-Process [[-Name] <string[]>] [<CommonParameters>]

Get-Process [[-Name] <string[]>] [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

Get-Process [<CommonParameters>]

PARÁMETROS
-ComputerName <string[]>

¿Requerido? false
¿Posición? Con nombre
¿Aceptar canalización? true (ByPropertyName)
Nombre de conjunto de parámetros Id Name InputObject
```

PowerShell – Ejemplos

Get-Eventlog , permite obtener los logs del sistema operativo.

Si se quiere obtener los 10 registros más recientes de log de Aplicaciones se puede usar: **Get-Eventlog -Logname Application -Newest 10**

```
Windows PowerShell
PS C:\> Get-Eventlog

cmdlet Get-EventLog en la posición 1 de la canalización de comandos
Proporcione valores para los parámetros siguientes:
LogName: Application
```

Index	Time	EntryType	Source	InstanceID	Message
389437	ago. 02 19:26	Error	SideBySide	3238068302	Error al generar el contexto de activación para...
389436	ago. 02 19:26	Error	SideBySide	3238068302	Error al generar el contexto de activación para...
389435	ago. 02 19:12	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389434	ago. 02 19:12	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389433	ago. 02 18:51	Error	.NET Runtime	1022	.NET Runtime version 4.0.30319.0 - Error al ini...
389432	ago. 02 18:50	Information	Edge	2147483904	[
389431	ago. 02 18:48	Information	Chrome	2147483904	[
389430	ago. 02 18:47	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389429	ago. 02 18:46	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389428	ago. 02 18:46	Information	Edge	2147483904	[
389427	ago. 02 18:39	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389426	ago. 02 18:39	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389425	ago. 02 18:28	Information	Microsoft-Windows...	10001	Ending session 0 started 2026-08-02T23:28:39.46...
389424	ago. 02 18:28	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389423	ago. 02 18:28	Information	Microsoft-Windows...	10000	Starting session 0 - 2026-08-02T23:28:39.4694006Z.
389422	ago. 02 18:28	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389421	ago. 02 18:24	Information	SecurityCenter	15	Updated Windows Defender status successfully to...
389420	ago. 02 18:24	Information	SecurityCenter	15	Updated Windows Defender status successfully to...
389419	ago. 02 17:55	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...

```
PS C:\> Get-Eventlog -Logname Application -Newest 10
```

Index	Time	EntryType	Source	InstanceID	Message
389438	ago. 02 19:37	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389437	ago. 02 19:26	Error	SideBySide	3238068302	Error al generar el contexto de activación para...
389436	ago. 02 19:26	Error	SideBySide	3238068302	Error al generar el contexto de activación para...
389435	ago. 02 19:12	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389434	ago. 02 19:12	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...
389433	ago. 02 18:51	Error	.NET Runtime	1022	.NET Runtime version 4.0.30319.0 - Error al ini...
389432	ago. 02 18:50	Information	Edge	2147483904	[
389431	ago. 02 18:48	Information	Chrome	2147483904	[
389430	ago. 02 18:47	Information	Software Protecti...	1073758208	El servicio de protección de software se progra...
389429	ago. 02 18:46	Information	Software Protecti...	3221241866	La migración de nivel inferior sin conexión se ...

PowerShell – Ejemplos

Canalización. Una canalización es una serie de comandos conectados por operadores de canalización “|” **conocido como PIPE**, normalmente escritos en una sola línea.

Get-Process | Select-Object -Property Name, significa que el resultado de obtener la lista de procesos en ejecución se limita mediante el segundo comando **Select-Object** a mostrar solamente la propiedad nombre.

Get-Process | Get-Member

```
PS C:\> Get-Process | Select-Object -Property Name

Name
----
AggregatorHost
ai
ai
AlienFXSubAgent
AppActions
ApplicationFrameHost
audiodg
AWCC.SCSUBAgent
AWCC.UCSubAgent
AWGameLibrary.SCSUBAgent
AWGameLibrary.UCSubAgent
AWPerformance.SCSUBAgent
AWPerformance.UCSubAgent
backgroundTaskHost
backgroundTaskHost
```

Get-Eventlog -Logname Application -Newest 5 | Select-Object -property EventId,Message,Timegenerated

```
PS C:\> Get-Eventlog -Logname Application -Newest 5 | Select-Object -property EventId,Message,Timegenerated

EventID TimeGenerated      Message
-----
16384 02/08/2026 19:45:08 El servicio de protección de software se programó correctamente para reiniciarse a las
16394 02/08/2026 19:44:37 La migración de nivel inferior sin conexión se realizó correctamente.
16384 02/08/2026 19:38:01 El servicio de protección de software se programó correctamente para reiniciarse a las
16394 02/08/2026 19:37:30 La migración de nivel inferior sin conexión se realizó correctamente.
78 02/08/2026 19:26:07 Error al generar el contexto de activación para "C:\Program Files (x86)\WinMerge\WinMer
```

PowerShell

OPERADORES	
Operadores de asignación	
=, +=, -=, *=, /=, %=, ++, --	Asigna, Quita uno o más valores a una variable.
Operadores de comparación	
-eq, -ne	Es igual, No es igual
-gt, -ge	Mayor que, Mayor o igual que
-lt, -le	Menor que, Menor o igual que
-like, -notlike	Es como, No es como (usa comodines)
-match, -notmatch	Busca una cadena usando expresiones regulares. Cuando la entrada es escalar, rellena la variable automática
-contains, -notcontains	Contiene, No contiene (coincidencia exacta)
-in, -notin	Aparece, No aparece en una colección de valores.
Operadores de tipo	
-is, isnot	Es, No es de un cierto tipo de objeto.
-as	Fuerza a tratar a un tipo de objeto como otro tipo de objeto.

PowerShell – Ejemplos

```
# =====  
# 1. MANIPULACIÓN DE ARCHIVO DE TEXTO PLANO (.txt)  
# =====  
  
# Definir la ruta del archivo  
$rutaTxt = "C:\TEMP\clase_sistemas_operativos.txt"  
  
Write-Host "--- TRABAJANDO CON ARCHIVO DE TEXTO ---" -ForegroundColor Cyan  
  
# A. CREACIÓN Y ESCRITURA INICIAL (Sobrescribe si ya existe)  
$contenidoInicial = "Tema 1: Arquitectura de Computadores e Hilos.`nTema 2: Gestión de Memoria en Linux.`nTema 3: Sistemas de Archivos NTFS."  
Set-Content -Path $rutaTxt -Value $contenidoInicial  
Write-Host "[OK] Archivo creado en: $rutaTxt" -ForegroundColor Green  
  
# B. ACCESO (Lectura)  
Write-Host "`nContenido actual del archivo:" -ForegroundColor Yellow  
Get-Content -Path $rutaTxt  
  
# C. MODIFICACIÓN (Reemplazar una palabra específica en el archivo)  
# Leemos el archivo en memoria, reemplazamos 'Linux' por 'Sistemas Unix' y guardamos de vuelta  
(Get-Content -Path $rutaTxt) -replace "Linux", "Sistemas Unix" | Set-Content -Path $rutaTxt  
Write-Host "`n[OK] Archivo modificado. Nuevo contenido:" -ForegroundColor Green  
Get-Content -Path $rutaTxt
```


PowerShell – Ejemplos

```
# =====
# 2. MANIPULACIÓN DE ARCHIVOS CSV (Objetos estructurados)
# =====

$rutaCsv = "C:\TEMP\notas_alumnos.csv"

Write-Host "--- TRABAJANDO CON ARCHIVO CSV ---" -ForegroundColor Cyan

# A. CREACIÓN Y EXPORTACIÓN (Creamos objetos PSCustomObject en memoria)
$alumnos = @(
    [PSCustomObject]@{ ID = 101; Nombre = "Ana Gomez"; Nota = 9.5; Estado = "Aprobado" }
    [PSCustomObject]@{ ID = 102; Nombre = "Carlos Perez"; Nota = 4.8; Estado = "Reprobado" }
    [PSCustomObject]@{ ID = 103; Nombre = "Lucia Diaz"; Nota = 8.0; Estado = "Aprobado" }
)
# El parámetro -NoTypeInfo evita que se guarde la cabecera de metadatos de .NET
$alumnos | Export-Csv -Path $rutaCsv -NoTypeInfo -Encoding Utf8
Write-Host "[OK] CSV exportado con éxito." -ForegroundColor Green

# B. ACCESO (Importación y lectura como objetos reales)
Write-Host "`nLeyendo datos del CSV (reconstruidos como objetos):" -ForegroundColor Yellow
$datosImportados = Import-Csv -Path $rutaCsv
$datosImportados | Format-Table -AutoSize

# C. MODIFICACIÓN (Actualizar la nota de Carlos Perez de 4.8 a 10.0 y recalcular su estado)
Write-Host "`nModificando registro de Carlos Perez..." -ForegroundColor Yellow
foreach ($alumno in $datosImportados) {
    if ($alumno.Nombre -eq "Carlos Perez") {
        $alumno.Nota = "10.0" # Modificación de propiedad
        $alumno.Estado = "Aprobado"
    }
}
# Guardamos los cambios de vuelta al disco duro
$datosImportados | Export-Csv -Path $rutaCsv -NoTypeInfo -Encoding Utf8
Write-Host "[OK] CSV actualizado. Datos finales:" -ForegroundColor Green
Import-Csv -Path $rutaCsv | Format-Table -AutoSize
```

PowerShell – Ejemplos

```
# =====
# 3. MANIPULACIÓN DE EXCEL (.xlsx) USANDO COM (Inter-Process Communication)
# =====

$rutaExcel = "C:\TEMP\reporte_arquitectura.xlsx"

Write-Host "--- TRABAJANDO CON EXCEL (.XLSX) ---" -ForegroundColor Cyan

try {
    # A. CREACIÓN Y APERTURA DE LA INSTANCIA DE EXCEL
    # Creamos un proceso invisible de Excel en segundo plano gerenciado por el S.O.
    $excel = New-Object -ComObject Excel.Application
    $excel.Visible = $false
    $excel.DisplayAlerts = $false

    # Creamos un nuevo libro y seleccionamos la primera hoja
    $workbook = $excel.Workbooks.Add()
    $worksheet = $workbook.Worksheets.Item(1)

    # B. ESCRITURA DE DATOS (Fila, Columna)
    $worksheet.Cells.Item(1, 1) = "Componente"
    $worksheet.Cells.Item(1, 2) = "Estado Físico"
    $worksheet.Cells.Item(2, 1) = "Memoria Caché L1"
    $worksheet.Cells.Item(2, 2) = "Operativo"

    # Guardamos el archivo y cerramos el libro
    $workbook.SaveAs($rutaExcel)
    $workbook.Close()
    Write-Host "[OK] Libro de Excel creado con éxito." -ForegroundColor Green

    # C. ACCESO Y MODIFICACIÓN
    Write-Host "`nAbriendo y modificando celda en Excel..." -ForegroundColor Yellow
    $workbook = $excel.Workbooks.Open($rutaExcel)
    $worksheet = $workbook.Worksheets.Item(1)

    # Modificamos el valor de la celda (2,2) de "Operativo" a "Mantenimiento preventivo"
    $worksheet.Cells.Item(2, 2) = "Mantenimiento preventivo1"
```

PowerShell – Ejemplos

```
# =====  
# 3. MANIPULACIÓN DE EXCEL (.xlsx) USANDO COM (Inter-Process Communication)  
# =====  
  
$rutaExcel = "C:\TEMP\reporte_arquitectura.xlsx"  
  
Write-Host "--- TRABAJANDO CON EXCEL (.XLSX) ---" -ForegroundColor Cyan  
  
try {  
    # A. CREACIÓN Y APERTURA DE LA INSTANCIA DE EXCEL  
    # Creamos un proceso invisible de Excel en segundo plano gerenciado por el S.O.  
    $excel = New-Object -ComObject Excel.Application  
    $excel.Visible = $false  
    $excel.DisplayAlerts = $false  
  
    # Creamos un nuevo libro y seleccionamos la primera hoja  
    $workbook = $excel.Workbooks.Add()  
    $worksheet = $workbook.Worksheets.Item(1)  
  
    # B. ESCRITURA DE DATOS (Fila, Columna)  
    $worksheet.Cells.Item(1, 1) = "Componente"  
    $worksheet.Cells.Item(1, 2) = "Estado Físico"  
    $worksheet.Cells.Item(2, 1) = "Memoria Caché L1"  
    $worksheet.Cells.Item(2, 2) = "Operativo"  
  
    # Guardamos el archivo y cerramos el libro  
    $workbook.SaveAs($rutaExcel)  
    $workbook.Close()  
    Write-Host "[OK] Libro de Excel creado con éxito." -ForegroundColor Green  
  
    # C. ACCESO Y MODIFICACIÓN  
    Write-Host "`nAbriendo y modificando celda en Excel..." -ForegroundColor Yellow  
    $workbook = $excel.Workbooks.Open($rutaExcel)  
    $worksheet = $workbook.Worksheets.Item(1)  
  
    # Modificamos el valor de la celda (2,2) de "Operativo" a "Mantenimiento preventivo"  
    $worksheet.Cells.Item(2, 2) = "Mantenimiento preventivo1"
```



Fin de la presentación